

drive.io MVP Enhancement Instructions: Pointer-Based Data Layer for Agentic Workflows

This document provides a comprehensive instruction set for an AI coding tool (e.g., Antigravity, Claude Code) to implement the pivot of drive.io from a simple agentic storage platform to a high-performance, pointer-based data layer for multi-agent systems.

Part 1: Current State Analysis

1.1 Product Summary

drive.io is currently a "Data Persistence Layer for AI Agents." It serves as a unified STDOUT and cloud drive for autonomous agents (like Moltbot and OpenClaw), allowing them to persist context, share artifacts, and handoff payloads via standard HTTP/S. It primarily functions as a "Stateless Memory" provider where agents can upload text/code/data and receive a shareable URL.

1.2 Current Landing Page Copy & Messaging

- **Hero Headline:** "Save Tokens. Expand Context for the Agent Swarm."
- **Subheadline:** "Stop wasting precious limits passing large datasets between agents. Programmatically offload files and context windows into secure, ephemeral storage instantly. Zero human authentication required."
- **Key Value Props:** Programmatic Persistence (zero-auth data URLs), Slash Token Usage (passing massive datasets between models), Provenance & Audit Logs (tracking AI actions).
- **Target Audience:** Autonomous agents, AI assistants in Moltbook, and developers building "swarm intelligence."

1.3 Current Feature Set

- **Clips API:** `POST /api/v1/clips` to save content and receive a URL.
 - **Agentic Protocol V1:** Standardized skill interface (`SKILL.md`) for agent consumption.
 - **Zero-Auth Data URLs:** Programmatically generated links for inter-agent handoff.
 - **Chrome Extension:** Manual capture of text/images to the cloud drive.
-

Part 2: MVP Feature Enhancement Instructions

2.1 Pointer/Reference System

Goal: Transition from simple "links" to a structured "Pointer" system that allows granular data access.

- **Implementation:**
 - **Data Model:** Update the `Clip` model to include `contentType` (json, csv, md, txt), `size`, and `checksum`.
 - **Pointer Format:** Standardize pointers as `dio://[pointer_id]`.
 - **Logic:** When data is uploaded, generate a unique `pointer_id`. Store the raw data in a way that supports range requests or structured querying (e.g., S3 for blobs, a document DB for JSON).
 - **Files to Modify:** `models/Pointer.ts`, `services/StorageService.ts`.

2.2 Agent-Friendly API (v2)

Goal: New REST endpoints for advanced data handoffs.

- **Endpoints:**
 - `POST /api/v2/pointers` : Upload data (JSON/File/Text) -> Returns `{ pointer_id, dio_uri, metadata }`.
 - `GET /api/v2/pointers/{id}` : Resolve pointer. Support query params for slicing:
 - `?select=field1,field2` (for JSON)
 - `?lines=1-100` (for Text/Code)
 - `?rows=0-50` (for CSV)
 - `PATCH /api/v2/pointers/{id}` : Append or update data at an existing pointer.
 - `GET /api/v2/pointers/{id}/schema` : Returns only the data structure (e.g., JSON schema or CSV headers).
- **Permissions:** Implement `scoped_tokens` that allow an agent to share a pointer with "read-only" or "limited-time" access.

2.3 MCP Integration

Goal: Allow agents to use drive.io natively via the Model Context Protocol.

- **Implementation:**
 - Create an MCP Server (Node.js/TypeScript).

- **Tools to expose:**
 - `create_pointer` : Uploads context/data.
 - `read_pointer` : Fetches data (with slicing support).
 - `get_pointer_metadata` : Gets schema/info without the full blob.
- **Resources:** Expose `dio://` URIs as MCP resources.
- **File Path:** `server/mcp/index.ts` .

2.4 A2A (Agent-to-Agent) Compatibility

Goal: Standardize pointer passing for frameworks like CrewAI and AutoGen.

- **Implementation:**
 - Update `SKILL.md` to include `resolve_pointer` tool definition.
 - Provide a Python SDK wrapper: `pip install drive-io-sdk` .
 - **Example Usage:** `agent_a.output = drive_io.create_pointer(large_data); agent_b.input = drive_io.resolve_pointer(agent_a.output, slice="summary")` .

2.5 Context Window Optimization

Goal: Help agents avoid "Context Bloat."

- **Endpoints:**
 - `POST /api/v2/pointers/{id}/summarize` : Uses a lightweight LLM (e.g., gpt-4o-mini) to generate a <500 token summary of the data at the pointer.
 - **Metadata Header:** Always include `X-DIO-Token-Estimate` in API responses to tell the agent how many tokens the current slice will consume.

2.6 Dashboard/Analytics

Goal: Visualize the value of the data layer.

- **Metrics to Track:**
 - `Total Pointers Created`
 - `Tokens Saved` : Calculated as `(Original Data Size in Tokens) - (Pointer Resolution Size in Tokens)` .
 - `Handoff Latency` : Time taken for Agent B to resolve Agent A's pointer.
- **UI Component:** A "Savings Calculator" on the user dashboard.

Part 3: Landing Page Copy Updates

3.1 Hero Section

- **Headline:** "The High-Performance Data Layer for Agentic Workflows."
- **Subheadline:** "Decouple data residency from agent reasoning. Pass 100MB datasets between agents for the cost of a single token. Stop context bloat, eliminate 'context rot,' and scale your AI swarm."
- **Primary CTA:** "Start Scaling for Free"
- **Secondary CTA:** "View API Docs"

3.2 New Feature Sections

- **Section 1: Pointer-Based Handoffs:** "Instead of passing massive data blobs, agents pass a `dio://` pointer. Resolve data on-demand, fetching only the specific rows or fields needed for the next step."
- **Section 2: Zero-Bloat Context:** "Our built-in slicing and summarization tools ensure your agents only 'see' what matters. Reduce token costs by up to 98% in complex multi-step pipelines."
- **Section 3: Framework Native:** "Native support for CrewAI, AutoGen, LangGraph, and OpenAI Swarm. One unified data layer for your entire agentic stack."

3.3 New "How It Works" Section

1. **Agent A Offloads:** Agent A generates a large dataset and calls `create_pointer()` .
2. **Pass the Pointer:** Agent A passes the lightweight `dio://` URI to Agent B.
3. **Agent B Queries:** Agent B queries the pointer for a specific slice (e.g., "just the executive summary") and continues the work.

3.4 Value Propositions (Updated)

- **Eliminate Context Rot:** Keep agents fresh by offloading historical data to the pointer layer.
 - **Framework Agnostic:** Move data seamlessly between a CrewAI agent and an AutoGen agent.
 - **Audit-Ready State:** Every pointer handoff is logged, creating a perfect trace of data flow through your swarm.
-

Part 4: Technical Implementation Details

4.1 Recommended Tech Stack

- **Backend:** FastAPI (Python) or NestJS (Node.js) for high-concurrency API.
- **Database:** PostgreSQL (Metadata) + Redis (Ephemeral Pointers) + S3 (Large Blobs).
- **Auth:** Stateless JWT with `agent_id` scoping.

4.2 API Schema (v2)

JSON

```
// POST /api/v2/pointers
{
  "data": "...", // String, JSON, or Multi-part file
  "ttl": 3600,    // Optional expiry in seconds
  "tags": ["financial-report", "q3-analysis"]
}

// GET /api/v2/pointers/{id}?select=summary,metrics
{
  "pointer_id": "dio_abc123",
  "data": {
    "summary": "Revenue up 20%...",
    "metrics": [1.2, 4.5, 3.1]
  },
  "metadata": {
    "total_size_tokens": 15420,
    "slice_size_tokens": 120,
    "savings_tokens": 15300
  }
}
```